Building a Local AI Assistant with LangGraph

Loubna El Ghazi
Advanced Artificial Intelligence / Generative AI
Practical Lab Report | 31 October 2026

1. Objective

The aim of this lab was to understand how LangGraph organizes an AI application into connected steps. Starting with a basic message-processing graph, we built a university assistant that classifies questions, uses tools, and remembers a conversation.

2. Tools and configuration

The implementation uses Python, LangChain, LangGraph, and Ollama. The model is **Qwen3.5:4b**, running locally instead of using the OpenAI API. It is accessed through ChatOllama at localhost:11434, with temperature set to 0, reasoning disabled, and a context window of 8192 tokens.

3. Main concepts

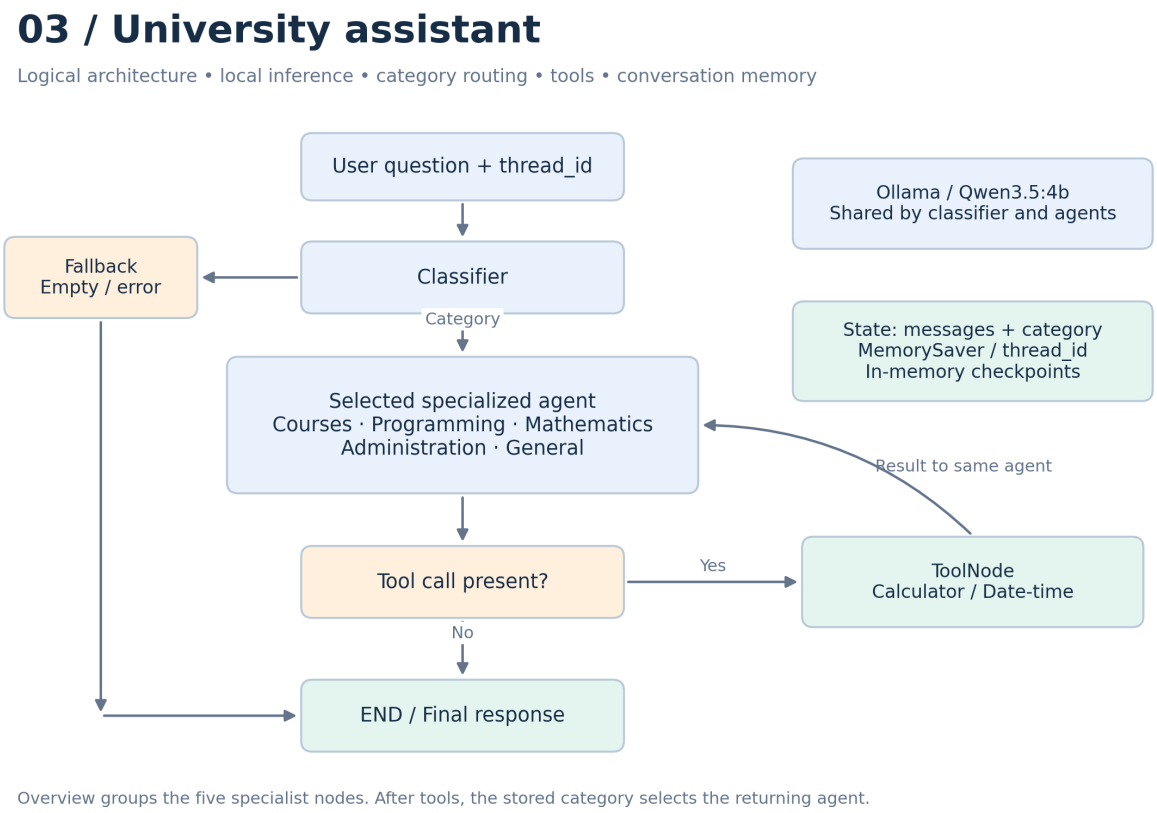
Concept	Role in the project
LangChain	Connects the model to messages, prompts, and tools.
LangGraph	Controls the workflow, shared state, routing, and memory.
LLM	Generates answers, classifies questions, and requests tools.
Node	A function that reads state, performs a task, and returns updates.
Edge	Defines which step runs next. A conditional edge selects a branch.
START / END	Mark the entry point and completion of graph execution.
State	Shares information between nodes, including questions, categories, and messages.

4. From a workflow to an agent

A fixed workflow follows a predefined sequence, such as analyzing a question and then answering it. In the tool-using workflow, the model can request a calculator, receive its result, and continue before giving a final answer.

5. University assistant architecture

The classifier routes each question to one of five specialists: courses, programming, mathematics, administration, or general assistance. These specialists share the same model but use different instructions. Empty questions and classification errors use a fallback response.



When a specialist requests a tool, ToolNode executes the calculator or date/time function. The result returns to the same specialist, which can then produce the answer. The calculator parses arithmetic expressions rather than using the unsafe eval() example.

6. Conversation memory

Messages are preserved with the add_messages reducer and MemorySaver. The thread_id identifies the conversation. Without checkpointing, the assistant did not recall the previously supplied name. With the same thread ID, it correctly recalled **Ahmed**. A different thread did not have that information.

This memory works by restoring previous messages as model context. It does not train the model, and it is lost when the notebook kernel restarts.

7. Results

The three-category router correctly classified all six evaluation questions. The recorded times include both classification and answer generation.

Question	Category	Time
Derivative of x^2	Mathematics	5.29 s
Solve 2x + 3 = 11	Mathematics	3.18 s
Reverse a Python list	Programming	10.25 s
Write a palindrome function	Programming	7.48 s
Convolutional neural network	Artificial intelligence	17.00 s
Explain overfitting	Artificial intelligence	23.61 s

Routing accuracy: 6/6 (100%). This result applies to the six tested examples. It does not measure the correctness of every generated answer or the general accuracy of the final assistant.

Functional checks

The tool trace for **125 × 38** showed a calculator call returning **4750**. The final assistant also used the calculator for **(15 + 27) × 3** and returned **126**. For the capital of Morocco, it answered **Rabat** without calling a tool. An unsupported import expression was rejected by the calculator.

The final demonstration selected the intended categories for the five sample university questions. It returned **4767** for 125 × 38 + 17 and displayed a fallback message for an empty question. The date/time function returned a timestamp, and the assistant displayed the date of the run.

8. Discussion and limitations

The router did not fail on the tested questions. More reliable routing could use structured output with a validated category list, exact matching, and additional ambiguous examples. The current evaluation uses substring matching.

The assistant has no official university database, so administrative answers remain general. The notebook also showed a warning about the date/time tool argument name. PNG graph rendering failed because the external rendering service was unreachable; Mermaid source remained available. These issues did not prevent the recorded assistant demonstrations.

9. Conclusion

This lab demonstrated how a local language model can work with LangGraph to route questions, execute tools, and maintain conversation history. The observed tests confirmed the main functions of the assistant. A larger evaluation and verified university data would be needed before using it as an institutional service.